ACRME Security and RBAC Guide

Azure Capacity Reservation Management Engine (ACRME)
Author: Vishnuvardhan Reddy — August 2026 (reconciled to Requirements Baseline v2.4, 7 September 2026)
Status: Prescriptive baseline for implementation and security review
Companion artifacts: `Reference-Material/rbac/custom_roles/*.json`, `Reference-Material/rbac/deploy_custom_roles.sh`, `Reference-Material/rbac/deploy_role_assignments.sh`

> **v2.4 reconciliation note (7 Sep 2026).** The authorization model — least-privilege UAMIs, isolated role-assignment authority, resource-group-scoped assignment, and the G-14 consent posture — is **unchanged** by Requirements Baseline v2.4. Two v2.4 items intersect this guide and are addressed inline: (a) **OPS-006/C-12 deterministic naming** — RBAC assignment scopes must bind to the deterministic RG/CRG/subscription names produced by OPS-006 (not ad-hoc names); see the new **Section 5.1**. (b) **CAP-023 regional + per-AZ CRG structure** — where a role is assigned at CRG scope, the target CRG set now includes the regional CRG (`crg-<env>-<region>-reg`) plus the per-AZ CRGs (`az1/az2/az3`); assignment stays at the narrowest resource-group/CRG scope as before. The seed-matrix governance (CAP-022) and reactive-discovery auto-create (CAP-024) are performed by the existing **ACRME Capacity Operator** identity within its approved provider resource groups — no new engine identity or broader scope is introduced.

---

1. Purpose and scope

This guide defines the complete authorization model for ACRME. It maps **every operation the engine performs** to the **exact Azure control-plane permissions** required, and it provides **least-privilege custom role definitions** for each functional identity. It is the authoritative reference for:

- Security review and threat modeling of the ACRME control plane.
- Provisioning the managed identities the engine runs under.
- Auditing what the engine can and cannot do in each managed subscription.
- Closing production gate **G-14** (consumer VM disassociation, Tier 3) with a customer-consented, resource-group-scoped identity rather than subscription-wide Virtual Machine Contributor.

Design principle — least privilege by construction. ACRME never runs as Owner, Contributor, or User Access Administrator. Each function runs under a dedicated identity holding a purpose-built custom role scoped to the narrowest resource container that still lets the function complete. Role assignment (a privileged operation) is isolated from capacity and VM mutation, and no engine identity may broaden its own rights.

What this guide does not cover. Data-plane secrets for Cosmos DB / Redis (handled by managed identity + Azure RBAC data roles, out of scope here beyond Section 12), network security groups, and Azure Policy authoring. Those are referenced where they intersect authorization.

---

2. Identity model

ACRME uses **User-Assigned Managed Identities (UAMI)**, one per function. UAMIs are preferred over system-assigned identities because they:

- Survive compute recreation (AKS node pool / Container Apps revision churn) without re-granting roles.
- Can be assigned before the workload exists, enabling infrastructure-as-code role assignment.
- Make the blast radius of each function explicit and independently revocable.

No stored client secrets. ACRME holds no service-principal client secrets or certificates. All Azure authentication is federated through managed identity token acquisition (`DefaultAzureCredential` → `ManagedIdentityCredential`). This eliminates the most common credential-leak class and removes secret-rotation from the operational burden.

2.1 Functional identities

#	Identity (UAMI)	Function	Custom role	Default scope
1	`id-acrme-reader`	Discovery, snapshotting, ARG queries, placement reads, reconciliation reads	**ACRME Reader**	All managed subscriptions (reader-only)
2	`id-acrme-capacity-operator`	CRG / CR create, update, resize, delete	**ACRME Capacity Operator**	Approved provider resource groups
3	`id-acrme-sharing-operator`	CRG sharing profile edits + approved role-assignment onboarding	**ACRME Sharing Operator**	Approved CRGs + onboarding scopes
4	`id-acrme-quota-operator`	Quota query and increase requests	**ACRME Quota Operator**	Approved quota scopes
5	`id-acrme-consumer-compute-operator`	**Tier 3** consumer VM disassociation (G-14, disabled by default)	**ACRME Consumer Compute Operator**	Explicit consumer resource groups only

2.2 Application (engine-API) roles

The five identities above are **Azure control-plane** identities. On top of them, the ACRME API enforces **application-level roles** that gate who may ask the engine to act. These map to the operational personas in the production-readiness review and are en-

forced in the API authorization layer (Entra ID app roles / groups), independent of the Azure identity the engine then uses to execute.

App role	May request	May not
DR Operator	Failover, failback, Tier 1, approved Tier 2	Role assignment, policy editing
Emergency Operator	Tier 3 request (after G-14 enablement)	Autonomous target selection
Policy Admin	Version and activate placement/transfer policy	Execute DR operations
Auditor	Read audit trail and evidence	Any mutation

Separation is deliberate: a DR Operator can drive the engine but cannot grant themselves or the engine new Azure rights; a Policy Admin shapes behavior but cannot execute it; an Auditor sees everything and changes nothing. See Section 10 (separation of duties).

---

3. Operation → permission matrix

This is the core of the guide. Each row is a concrete engine operation, the service that owns it, the exact Azure action(s) required, and the identity that carries them. Actions are Azure Resource Manager control-plane operations unless noted.

3.1 Discovery and read operations — ACRME Reader

Operation	Owning service	Required actions	Notes
List/read CRGs	RegionalSnapshotService	`Microsoft.Compute/capacityReservationGroups/read`	
List/read capacity reservations	RegionalSnapshotService	`Microsoft.Compute/capacityReservations/read`	Includes `instanceView` for utilization
Read VM → CRG association state	RegionalSnapshotService	`Microsoft.Compute/virtualMachines/read`	Reads `capacityReservationGroup` property
Read VMSS association state	RegionalSnapshotService	`Microsoft.Compute/virtualMachineScaleSets/read`	
Query quota / usage	QuotaValidationService	`Microsoft.Quota/quotas/read`, `Microsoft.Quota/usages/read`, `Microsoft.Capacity/resourceProviders/locations/serviceLimits/read`	
Read SKU / zone capability	ZoneMappingService	`Microsoft.Compute/locations/vmSizes/read`, `Microsoft.Compute/skus/read`	Logical→physical zone mapping
Resource Graph fleet query	AzureResourceGraphClient	`Microsoft.ResourceGraph/resources/read` + ARM read on target resources	ARG returns only resources the identity can read; 403 if no readable subscriptions
Read subscription / RG metadata	RegionalSnapshotService	`Microsoft.Resources/subscriptions/read`, `Microsoft.Resources/subscriptions/resourceGroups/read`	
Read operation status (async LRO)	OperationTracker	`Microsoft.Compute/locations/operations/read`, `Microsoft.`	Poll long-running operations

Operation	Owning service	Required actions	Notes
		`crosoft.Resources/deployments/read`	
Read role assignments (audit self)	AuditService	`Mi-crosoft.Authorization/roleAssignments/read`, `Mi-crosoft.Authorization/roleDefinitions/read`	Verify least-privilege posture

> **Placement** is read-only at the Azure layer: PlacementEngine consumes RegionalSnapshotService + QuotaValidationService reads and emits a recommendation. It requires **no write action**. Execution of a recommendation is a separate, explicitly authorized mutation performed by the Capacity or Consumer Compute identity.

3.2 Capacity reservation lifecycle — ACRME Capacity Operator

Operation	Owning service	Required actions	Notes
Create CRG	CapacityTransferService	`Microsoft.Compute/capacityReservationGroups/write`	Scoped to approved provider RG
Update / delete CRG	CapacityTransferService	`Microsoft.Compute/capacityReservationGroups/write`, `.../capacityReservationGroups/delete`	
Create CR	CapacityTransferService	`Microsoft.Compute/capacityReservations/write`	
Resize CR quantity (Tier 1/2 transfer core)	CapacityTransferService	`Microsoft.Compute/capacityReservations/write`	Increase = new capacity commit; decrease = release. Same action, different `capacity` value
Delete CR	CapacityTransferService	`Microsoft.Compute/capacityReservations/delete`	Only after zero associated VMs
Read-back after write	CapacityTransferService	`.../capacityReservationGroups/read`, `.../capacityReservations/read`	Confirm converged state

The Capacity Operator role **explicitly excludes** `capacityReservationGroups/share/action` (a `NotAction`) — enabling sharing is a distinct, more privileged operation isolated to the Sharing Operator. It also holds **no** `virtualMachines/write`, so it can never mutate consumer VMs.

3.3 Sharing (cross-subscription / cross-tenant) — ACRME Sharing Operator

Operation	Owning service	Required actions	Notes
Enable / edit CRG sharing profile	SharingManagement-Service	`Microsoft.Compute/capacityReservationGroups/share/action`	Provider-owner side; CRG sharing is public preview — use CLI/REST with `api-version=2024-03-01`
Read CRG being shared	SharingManagement-Service	`Microsoft.Compute/capacityReservationGroups/read`	
Grant consumer read+deploy on shared CRG	SharingManagement-Service	`Microsoft.Authorization/roleAssignments/write`, `Microsoft.Authorization/roleAssignments/read`, `Microsoft.Authorization/roleDefinitions/read`	The **only** engine identity permitted role-assignment write, and only at approved onboarding scopes
Consumer-side deploy against shared CRG	(consumer subscription)	`Microsoft.Compute/capacityReservationGroups/read`, `.../capacityReservations/read`, `.../capacityReservationGroups/deploy/action`	Granted to the consumer, not held by the engine

The Sharing Operator carries `roleAssignments/write` — the single most sensitive right in the model — so it is deliberately **fenced**: two `NotActions` block `capacityReservations/write` and `capacityReservations/delete`, meaning this identity can onboard consumers and edit sharing profiles but can never create, resize, or delete the underlying reservations. This enforces separation between who can share capacity and who can change capacity.

3.4 Quota management — ACRME Quota Operator

Operation	Owning service	Required actions	Notes
Read quota / usage	QuotaValidationService	`Microsoft.Quota/quotas/read`, `Microsoft.Quota/quotaRequests/read`, `Microsoft.Quota/usages/read`	
Submit adjustable quota increase	QuotaValidationService	`Microsoft.Quota/quotas/write`, `Microsoft.Quota/register/action`	Programmatic increase for supported SKUs/regions
Read/write service limits	QuotaValidationService	`Microsoft.Capacity/resourceProviders/locations/serviceLimits/{read,write}`, `.../serviceLimitsRequests/read`	Compute service-limit path
File support ticket (non-adjustable)	QuotaValidationService	`Microsoft.Support/supportTickets/write`, `Microsoft.Support/supportTickets/read`	Fallback when quota is not self-service

The Quota Operator holds **no compute write and no VM write** — it can only query and request quota. A quota increase never implicitly creates a reservation; that remains a separate Capacity Operator action.

3.5 Consumer VM disassociation (Tier 3, G-14) — ACRME Consumer Compute Operator

> **Disabled by default.** This identity and its role assignment are gated behind `engine_mode` and are **not** provisioned until gate G-14 is formally closed with customer consent. The role JSON exists and is validated, but `deploy_role_assignments.sh` leaves its assignment commented out.

Operation	Owning service	Required actions	Notes
Read VM association state	VMDisassociationService	`Microsoft.Compute/virtualMachines/read`	
Disassociate VM from CRG (Path B, in-place)	VMDisassociationService	`Microsoft.Compute/virtualMachines/write`	Clears `capacityReservationGroup`; requires VM stop-deallocate first on many SKUs
Deallocate / restart VM (Path A fallback)	VMDisassociationService	`Microsoft.Compute/virtualMachines/deallocate/action`, `Microsoft.Compute/virtualMachines/start/action`	Only when in-place update is unsupported
Read-back converged state	VMDisassociationService	`Microsoft.Compute/virtualMachines/read`	

Three `NotActions` fence this role hard: `virtualMachines/delete`, `virtualMachineScaleSets/write`, and `virtualMachineScaleSets/delete`. The engine can move a VM off a reservation but can **never delete a customer VM or touch scale sets**. Scope is restricted to **explicitly enumerated consumer resource groups** — never a subscription. See Section 11 for the full G-14 consent and revocation model.

3.6 Cross-cutting operations

Operation	Identity	Required actions	Notes
Reconciliation (drift detect)	Reader	read actions in Section 3.1	Detection is read-only
Reconciliation (drift remediate)	Capacity / Consumer Compute	corresponding write action in Section 3.2 / Section 3.5	Remediation reuses the mutation identity, still least-privilege
Emergency capacity transfer (Tier 1/2)	Capacity Operator	`capacityReservations/write`	Tier boundaries enforced in app layer, not Azure RBAC
DR failover / failback	Capacity Operator (+ Reader)	`capacityReservations/write`, reads	Orchestrated by app-role DR Operator
`engine_mode` transition	(no Azure action)	—	State in Cosmos DB; gated by Policy Admin app role
Audit event write	(data plane)	Cosmos DB data-role, not ARM	See Section 12

---

4. Custom role catalog

Five custom roles are defined as JSON under `Reference-Material/rbac/custom_roles/`. Each uses placeholder tokens (`<SUBSCRIPTION_ID>`, `<PROVIDER_RG_ID>`, etc.) substituted at deploy time by `deploy_custom_roles.sh`.

Role file	Actions	NotActions	Assignable scope intent
`acrme_reader.json`	19 read actions	—	Managed subscriptions
`acrme_capacity_operator.json`	13	1 (`share/action`)	Provider resource groups
`acrme_sharing_operator.json`	10 (incl. `share/action` + `roleAssignments/write`)	2 (CR write/delete)	Approved CRGs + onboarding scopes
`acrme_quota_operator.json`	13 (Quota + Capacity + Support)	—	Approved quota scopes
`acrme_consumer_compute_operator.json`	7	3 (VM delete, VMSS write/delete)	Explicit consumer RGs only

Design notes:

- **No wildcards on write.** Read roles may use narrow wildcards for convenience; every write/delete/action is explicitly enumerated so the role cannot silently acquire new mutation rights when Azure adds resource types.
- **`NotActions` as guardrails, not just least-privilege.** They encode invariants (Capacity can't share; Sharing can't change reservations; Consumer Compute can't delete VMs) that survive future edits to the `Actions` list.
- **`assignableScopes`** are set to the narrowest management-group/subscription that must host the role definition; assignment scope is narrowed further at assignment time (Section 5).

---

5. Scope and assignment guidance

Two-level narrowing. A custom role has (a) `assignableScopes` — where the definition may be assigned — and (b) the assignment scope — where an identity actually receives it. Always set (b) tighter than (a).

Identity	Definition scope	Assignment scope (target)
Reader	Management group / subscription	Each managed subscription (reader)
Capacity Operator	Subscription	Individual approved provider **resource groups**
Sharing Operator	Subscription	Specific **CRG resource IDs** + onboarding RG
Quota Operator	Subscription	Subscription or RG hosting quota
Consumer Compute Operator	Subscription	Enumerated consumer **resource groups** only

`deploy_role_assignments.sh` assigns each role to its UAMI at the narrowest scope and prints the resulting assignment IDs for audit capture. Never assign a mutation role at subscription scope when a resource-group scope suffices.

5.1 Deterministic naming and per-AZ CRG scopes (v2.4 — OPS-006, C-12, CAP-023)

Baseline v2.4 pins a **deterministic naming convention** for resource groups, CRGs, and subscriptions (**OPS-006**), backed by a uniqueness **counter (C-12)**. This does not change what rights are granted or how narrow the scope is — it changes only how the scope's **resource identity** is derived. Two practical rules:

1. **Bind assignment scopes to OPS-006 names, not ad-hoc names.** Assignment scopes (RG paths, CRG resource IDs) must reference the deterministic names emitted by OPS-006 rather than hand-picked strings. `deploy_role_assignments.sh` parameterises the RG/CRG names (`PROVIDER_CRG_RG` , etc.) precisely so these can be set to the OPS-006-generated values; keep those variables sourced from the naming output so assignments and resources never drift. Deterministic names also make **audit** trivial — the same inputs always yield the same, collision-free scope string.
2. **CRG-scoped assignments now span a regional + per-AZ CRG set (CAP-023).** Each environment carries one **regional** CRG (`crg-<env>-<region>-reg`) plus one CRG **per availability zone** (`az1/az2/az3`). Where a role (e.g. **ACRME Capacity Operator** or **ACRME Sharing Operator**) is assigned at CRG scope, enumerate the regional CRG **and** the per-AZ CRGs for that environment — still at the narrowest CRG/RG scope, never widened to the subscription. Because the names are deterministic, the per-AZ CRG scope list is derivable from the environment/region inputs without lookup.

No new identity or broader scope. Seed-matrix governance (**CAP-022**) and reactive-discovery auto-create (**CAP-024**) are ordinary CRG/CR create/update operations performed by the existing **ACRME Capacity Operator** within its approved provider resource groups. The product-team **budget-governance** approval for raising a seed reservation above 0 is an **application-layer** gate (an engine-API approval), not an Azure RBAC grant — it does not require any change to the control-plane roles here.

---

6. Cross-subscription and cross-tenant considerations

- **Cross-subscription sharing.** CRG sharing lets a provider subscription expose reservations to consumer subscriptions in the same tenant. The engine's Sharing Operator performs `share/action` on the provider side and grants the consumer identity read+deploy at the CRG scope. No engine identity needs standing rights in the consumer subscription for deployment — the consumer deploys under its own identity.
- **Cross-tenant.** Where consumers are in a separate tenant, use **Azure Lighthouse** delegated resource management to project the ACRME reader/operator UAMIs into the customer tenant at a delegated scope, rather than creating guest service principals. Lighthouse delegations are auditable and revocable by the customer, aligning with the G-14 consent posture.
- **ARG across tenants** returns only resources visible to the calling identity's delegated scope; expect partial fleet views and design reconciliation to treat unseen subscriptions as "not managed" rather than "drifted."

---

7. Deployment

```
# 1. Create/update the five custom role definitions (substitutes placeholders from
env)
export SUBSCRIPTION_ID="<sub>"
export PROVIDER_RG_ID="/subscriptions/<sub>/resourceGroups/<rg>"
bash Reference-Material/rbac/deploy_custom_roles.sh

# 2. Assign roles to the UAMIs at the narrowest scope (Tier 3 stays commented until
G-14)
export READER_UAMI_PRINCIPAL_ID="<...>"   # etc. per identity
bash Reference-Material/rbac/deploy_role_assignments.sh
```

Both scripts are idempotent (`az role definition create` → falls back to `update` ; `az role assignment create` is safe to re-run). Capture stdout as the authorization audit record.

---

8. Least-privilege verification

After deployment, verify no identity exceeds its intended rights:

```
# Enumerate every assignment for an engine UAMI
az role assignment list --assignee <UAMI_PRINCIPAL_ID> --all -o table
# Confirm the definition contains no unexpected Actions
az role definition list --name "ACRME Capacity Operator" --query "[].permissions" -o j
son
```

Acceptance: each UAMI holds exactly one ACRME custom role, at the expected scope, with no built-in Owner/Contributor/UAA assignment anywhere in its assignment list.

---

9. G-14 credential and consent model (Tier 3)

Gate **G-14** blocks autonomous consumer VM disassociation in production. Closure requires **all** of:

1. **Explicit customer consent** — a signed authorization naming the exact consumer resource groups and the exact operations (disassociate, and Path-A deallocate/start) the engine may perform.
2. **A narrow custom role** — `ACRME Consumer Compute Operator`, scoped to only the consented resource groups, with the three `NotActions` intact. **Never** subscription-wide Virtual Machine Contributor.
3. **Tested revocation** — a demonstrated, timed revocation path (remove the role assignment; confirm the engine receives 403 within the token TTL) captured as evidence.
4. **`engine_mode` gate** — Tier 3 remains inert unless the engine is explicitly placed in the mode that enables it; the app-role Emergency Operator submits the request, the engine never self-selects targets.

Until 1–4 are satisfied, `deploy_role_assignments.sh` leaves the Consumer Compute assignment commented out and the engine treats Tier 3 as unavailable.

---

10. Separation of duties

Concern	Held by	Explicitly denied to
Change capacity (CR write/delete)	Capacity Operator	Sharing Operator (NotAction), Reader, Quota
Share capacity + onboard consumers (role assignment)	Sharing Operator	Capacity, Reader, Quota, Consumer Compute
Mutate consumer VMs	Consumer Compute Operator (G-14)	Capacity, Sharing, Reader, Quota
Request quota	Quota Operator	everyone else
Grant Azure rights (`roleAssignments/write`)	Sharing Operator only, fenced to onboarding scopes	all other engine identities
Drive DR / Tier operations	DR Operator (app role)	Policy Admin, Auditor
Author policy	Policy Admin (app role)	DR / Emergency Operators
Observe everything	Auditor (app role)	— (read-only)

No single identity can both **grant rights** and **change capacity/VMs**. The only identity with `roleAssignments/write` cannot itself change reservations or VMs, so it cannot self-escalate into a capacity or compute mutation.

---

11. Break-glass and revocation

- **Break-glass** capacity operations use a separately-approved, time-boxed Privileged Identity Management (PIM) activation of the relevant ACRME custom role by a human operator — not a standing assignment and not a broader built-in role. Every activation is logged.
- **Kill switch.** Removing a UAMI's single role assignment fully revokes that function within the token TTL; because there are no stored secrets, there is nothing else to rotate or revoke.
- **Tier 3 emergency stop.** Setting `engine_mode` out of the Tier-3-enabled state disables consumer VM mutation at the application layer even before the Azure assignment is removed — defense in depth.

---

12. Secret-free and data-plane posture

- **Control plane:** managed identity only; no client secrets or certificates in code, config, key vault references for engine auth, or CI.
- **Data plane:** Cosmos DB and Redis are accessed via the engine's managed identity using Azure RBAC **data** roles (e.g., Cosmos DB Built-in Data Contributor) scoped to the ACRME database — not account keys. This keeps the "no stored secrets" invariant end to end.
- **Tokens:** all Azure tokens are short-lived and acquired at runtime; nothing durable is persisted.

---

13. Mapping to the production-readiness RBAC matrix

This guide operationalizes the RBAC matrix in the Production Readiness Review (Section 36). The five UAMIs here correspond 1:1 to the "ACRME * UAMI" rows; the four app roles correspond to the DR Operator / Emergency Operator / Policy Admin / Auditor rows. The G-14 closure model here is the concrete realization of that document's "customer-consented UAMI with resource-group-scoped custom rights" guidance.

---

14. Summary

ACRME runs under five narrowly-scoped managed identities, each with a purpose-built custom role, plus four application roles that gate who may drive the engine. No identity is an Owner, Contributor, or User Access Administrator; no secrets are stored; role-assignment authority is isolated from capacity and VM mutation; and the highest-risk operation (Tier 3 consumer VM disassociation) is fenced behind customer consent, a resource-group-scoped role, tested revocation, and an `engine_mode` gate. The custom role JSONs and deployment scripts in `Reference-Material/rbac/` make this model directly deployable and auditable.